

QUEUE QNO:1

QUESTION:

Problem Description

The Monk recently learnt about priority queues and requested his teacher for a fun challenge to solve. As a result, his teacher devised a simple task. He now possesses A , which is an integer array. He wishes to discover the product of the greatest, second largest, and third largest integers in the range $[1, i]$ for each index i .

Note: Two numbers can be the same value-wise but they should be distinct index-wise.

Constraints:

$1 \leq N \leq 100000$

$0 \leq A[i] \leq 1000000$

Input:

The first line contains an integer N , denoting the number of elements in the array A .

The next line contains N space separated integers, each denoting the i th integer of the array A .

Output:

Print the answer for each index in each line. If there is no second largest or third largest number in the array A upto that index, then print "-1", without the quotes.

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5
1 2 3 4 5
```

EXPECTED OUTPUT

```
-1
-1
6
24
60
```

Test Case 2

INPUT (STDIN)

```
8
3 7 1 2 4 7 5 9
```

EXPECTED OUTPUT

```
-1
-1
21
42
84
196
245
441
```

✓ Mandatory Test Cases

Test Case 1

KEYWORD

```
if(biggest<big)
```

Test Case 2

KEYWORD

```
if(a[i]>biggest)
```

CYCLOMATIC COMPLEXITY

10

TOKEN COUNT

300

NLOC

62

Source Code:

```
#include <stdio.h>
long long a[100005];

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;
    long long biggest = -1;
    long long big = -1;
    long long third = -1;

    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]);
        if(a[i]>biggest) {
            third = big;
            big = biggest;
            biggest = a[i];
        }
        else if (a[i] > big) {
            third = big;
            big = a[i];
        }
        else if (a[i] > third) {
            third = a[i];
        }

        if(biggest<big) {
            long long temp = biggest;
            biggest = big;
            big = temp;
        }

        if (i < 2) {
            printf("-1\n");
        } else {
            long long product = biggest * big * third;
            printf("%lld\n", product);
        }
    }
    return 0;
}
```

Output:

```
5
1 2 3 4 5
-1
-1
6
24
60
```

```
8
3 7 1 2 4 7 5 9
-1
-1
21
42
84
196
245
441
```